Sliding Window + Two Pointers: Constant Window Approach

The sliding window technique, when combined with the two-pointer approach, is a powerful method for solving problems that involve subarrays or substrings within a fixed range. The constant window approach maintains a window of a specific size and slides it across the array or string. This editorial details a method to efficiently compute the sum of elements within such a window by leveraging the properties of the sliding window technique.

Concept

The goal is to calculate the sum of elements within a fixed-size window as it moves across the array. A brute-force method would involve recalculating the sum of the window each time it moves, which would lead to a time complexity of O(n * k), where n is the number of elements in the array, and k is the size of the window.

However, a more efficient approach involves updating the sum incrementally. As the window slides from left to right, the sum of the current window can be derived from the sum of the previous window by subtracting the element that is leaving the window (on the left) and adding the element that is entering the window (on the right). This reduces the time complexity to O(n).

Steps

1. Initialize two pointers: left and right. The left pointer marks the beginning of the window, and the right pointer marks the end.
2. Calculate the sum of the initial window.
3. Slide the window by incrementing both left and right pointers. Update the sum by subtracting the element at the left pointer and adding the element at the right pointer.
4. Continue this process until the window has traversed the entire array.

Pseudocode

C++Java

```
#include<bits/stdc++.h>

using namespace std;


void slidingWindowSum(const vector<int>& arr, int k) {
   int n = arr.size();
   int sum = 0;

   // Calculate the sum of the initial window
   for (int i = 0; i < k; ++i) {
      sum += arr[i];
   }
```

```
    // Print the sum of the first window
    cout << "Sum of window 1: " << sum << endl;

    // Slide the window across the array
    for (int i = k; i < n; ++i) {
        sum = sum - arr[i - k] + arr[i]; // Update the sum
        cout << "Sum of window " << i - k + 2 << ": " << sum << endl;
    }
}

int main() {
    vector<int> arr = {1, 3, 2, 6, 4, 8, 5};
    int k = 3;
    slidingWindowSum(arr, k);
    return 0;
}
```

Conclusion

The constant window approach to solving sliding window problems provides a significant optimization over recalculating the sum for each window position. By simply updating the sum based on the elements that leave and enter the window, the time complexity is reduced to linear time, making it an efficient solution for large input sizes.

Longest Subarray or Substring Explanation

The problem statement revolves around finding the longest subarray or substring from a given array or string, where the sum of the elements is less than or equal to a given value K. This approach is widely used in coding interviews and is based on the sliding window technique.

Initially, you might start with a brute-force approach where you generate all possible subarrays or substrings and then check each one to see if it meets the condition. However, this is inefficient and can be optimized using the sliding window technique.

Sliding Window Technique

The sliding window technique helps in optimizing the process by maintaining a window that expands or shrinks based on the conditions. Here's how it works:

Step-by-Step Process:

1. **Initialize Pointers:** Start with two pointers, L (left) and R (right), both set to the beginning of the array or string. Also, initialize variables sum and maxLength to zero.
2. **Expand the Window:** Start expanding the window by moving the R pointer. Add the element at the R pointer to sum.
3. **Check the Condition:** If the sum is less than or equal to K, check if the current window size (R - L + 1) is the largest so far. If it is, update maxLength.
4. **Shrink the Window:** If the sum exceeds K, start shrinking the window by moving the L pointer to the right. Subtract the element at the L pointer from sum and continue this process until the sum is less than or equal to K again.
5. **Continue Until the End:** Repeat the above steps until the R pointer reaches the end of the array or string.
6. **Result:** The value in maxLength at the end of the process will be the length of the longest subarray or substring that satisfies the condition.

Dry Run

C++Java

```
#include<bits/stdc++.h>

using namespace std;


// Function to find the longest subarray with sum <= K

int longestSubarrayWithSum(vector<int>& arr, int K) {

   int n = arr.size();

   int maxLength = 0; // Initialize the maximum length to 0

   int sum = 0;      // Initialize the current sum to 0

   int left = 0;     // Left pointer for the sliding window


   // Iterate over the array using the right pointer

   for (int right = 0; right < n; right++) {

      sum += arr[right]; // Add the current element to the sum


      // Shrink the window from the left if sum exceeds K

      while (sum > K) {

         sum -= arr[left]; // Subtract the leftmost element from the sum

         left++;         // Move the left pointer to the right
```

```
        }

        // Update the maximum length if the current window is valid
        maxLength = max(maxLength, right - left + 1);
    }

    return maxLength; // Return the maximum length of the valid subarray
}

int main() {
    vector<int> arr = {2, 5, 1, 7, 10}; // Example array
    int K = 14; // Example value of K

    // Find and print the length of the longest subarray with sum <= K
    int result = longestSubarrayWithSum(arr, K);
    cout << "The longest subarray length with sum <= " << K << " is: " << result << endl;

    return 0;
}
```

The above code demonstrates the sliding window approach. The key is to balance the window's expansion and contraction to ensure that the sum of the elements inside the window is always less than or equal to K.

This method is much more efficient than the brute-force approach and is commonly used in problems involving subarrays or substrings with specific conditions.

Counting the Number of Subarrays with a Given Sum

In some sliding window problems, the task is not just to find the minimum or maximum length of a subarray, but to count the number of subarrays that satisfy a particular condition. This editorial focuses on such problems, where the goal is to determine the number of subarrays with a given sum, specifically for binary arrays.

Consider the problem of counting the number of subarrays that sum up to 'k'. The approach involves calculating the number of subarrays with at most K ones and then subtracting the number of subarrays with at most K-1 ones. The difference gives the number of subarrays with exactly K ones.

Method

1. Define a helper function that counts the number of subarrays with at most K ones. If K is less than 0, the result is 0.
2. Iterate through the array using two pointers. Adjust the window size to ensure the sum of the window is at most K. Incrementally add the number of valid subarrays to the result.
3. Subtract the result of atMost(K-1) from atMost(K) to get the number of subarrays with exactly K ones.

Dry Run

C++Java

```
#include<bits/stdc++.h>
using namespace std;

class Solution {
public:
  // Function to find the number of subarrays with exactly sum S
  int numSubarraysWithSum(vector<int>& A, int S) {
    return atMost(A, S) - atMost(A, S - 1);
  }

private:
  // Helper function to find the number of subarrays with at most sum S
  int atMost(vector<int>& A, int S) {
    if (S < 0) return 0;
    int res = 0, i = 0;

    // Sliding window approach to count subarrays
    for (int j = 0; j < A.size(); j++) {
    // Include A[j] in the current window
      S -= A[j];
      while (S < 0) {
      // Shrink the window if the sum exceeds S
        S += A[i++];
      }
```

```
            // Count all subarrays ending at j
            res += j - i + 1;
        }

        return res;
    }
};

int main() {
    Solution sol;
    vector<int> A = {1, 0, 1, 0, 1}; // Example array
    int S = 2; // Desired sum
    int result = sol.numSubarraysWithSum(A, S);
    cout << "Number of subarrays with sum " << S << ": " << result << endl;
    return 0;
}
```

Conclusion

By leveraging the difference between the number of subarrays with at most K ones and those with at most K-1 ones, it is possible to efficiently calculate the number of subarrays with exactly K ones. This method provides a concise and powerful solution to a class of sliding window problems.

36

Maximum Points You Can Obtain from Cards

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to calculate the maximum
    score after picking k cards*/
    int maxScore(vector<int>& cardScore, int k) {
        int lSum = 0, rSum = 0, maxSum = 0;
```

```
    // Calculate the initial sum of the first k cards
    for (int i = 0; i < k; i++) {
        lSum += cardScore[i];

        /* Initialize maxSum with the
        sum of the first k cards*/
        maxSum = lSum;
    }

    //Initialize rightIndex to iterate array from last
    int rightIndex = cardScore.size() - 1;

    for (int i = k - 1; i >= 0; i--) {

        // Remove the score of the ith card from left sum
        lSum -= cardScore[i];

        /* Add the score of the card
        from the right to the right sum*/
        rSum += cardScore[rightIndex];

        // Move to the next card from the right
        rightIndex--;

        // Update maxSum with the maximum sum found so far
        maxSum = max(maxSum, lSum + rSum);
    }

    // Return the maximum score found
    return maxSum;
}
```

```
};

int main() {
    vector<int> nums = {1, 2, 3, 4, 5, 6};

    // Create an instance of the Solution class
    Solution sol;

    int result = sol.maxScore(nums, 3);

    // Output the maximum score
    cout << "The maximum score is:\n";
    cout << result << endl;

    return 0;
}
```

Longest Substring Without Repeating Characters

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestNonRepeatingSubstring(string &s) {

        // Length of the input string
        int n = s.size();

        //Variable to store max length
        int maxLen = 0;
```

```
/* Iterate through all possible
starting points of the substring*/
for (int i = 0; i < n; i++) {

    /* Hash to track characters in
    the current substring window*/
    // Assuming extended ASCII characters
    vector<int> hash(256, 0);

    for (int j = i; j < n; j++) {

        /* If s[j] is already in the
        current substring window*/
        if (hash[s[j]] == 1) break;

        /* Update the hash to mark s[j]
        as present in the current window*/
        hash[s[j]] = 1;

        /* Calculate the length of
        the current substring*/
        int len = j - i + 1;

        /* Update maxLen if the current
        substring length is greater*/
        maxLen = max(maxLen, len);
    }
}

// Return the maximum length
return maxLen;
```

```
    }
};

int main() {
    string input = "cadbzabcd";

    //Create an instance of Solution class
    Solution sol;

    int length = sol.longestNonRepeatingSubstring(input);

    //Print the result
    cout << "Length of longest substring without repeating characters: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the longest substring
    without repeating characters*/
    int longestNonRepeatingSubstring(string& s) {
        int n = s.size();

        // Assuming all ASCII characters
        int HashLen = 256;

        /* Hash table to store last
        occurrence of each character*/
```

```
int hash[HashLen];

/* Initialize hash table with
-1 (indicating no occurrence)*/
for (int i = 0; i < HashLen; ++i) {
   hash[i] = -1;
}

int l = 0, r = 0, maxLen = 0;
while (r < n) {

   /* If current character s[r]
   is already in the substring*/
   if (hash[s[r]] != -1) {

      /* Move left pointer to the right
      of the last occurrence of s[r]*/
      l = max(hash[s[r]] + 1, l);
   }

   // Calculate the current substring length
   int len = r - l + 1;

   // Update maximum length found so far
   maxLen = max(len, maxLen);

   /* Store the index of the current
   character in the hash table*/
   hash[s[r]] = r;

   // Move right pointer to next position
```

```
            r++;
        }

        // Return the maximum length found
        return maxLen;
    }
};

int main() {
    string s = "cadbzabcd";

    // Create an instance of the Solution class
    Solution sol;

    int result = sol.longestNonRepeatingSubstring(s);

    // Output the maximum length
    cout << "The maximum length is:\n";
    cout << result << endl;

    return 0;
}
```

Max Consecutive Ones III

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the length of the
    longest substring with at most k zeros*/
    int longestOnes(vector<int>& nums, int k) {
```

```
// Length of the input array
int n = nums.size();

//Maximum length of the substring
int maxLen = 0;

/* Variable to count the number
of zeros in the current window*/
int zeros = 0;

/* Iterate through all possible
starting points of the substring*/
for (int i = 0; i < n; i++) {
    zeros = 0;

    /* Expand the window from starting
    point i to the end of the array*/
    for (int j = i; j < n; j++) {
        if (nums[j] == 0) {

            /* Increment zeros count
            when encountering a zero*/
            zeros++;
        }

        /* If zeros count is within the
        allowed limit (k), update maxLen*/
        if (zeros <= k) {

            // Calculate the length of substring
```

```
                int len = j - i + 1;
                maxLen = max(maxLen, len);
            } else break;
        }
    }

    //Return the maximum length
    return maxLen;
  }
};

int main() {
  vector<int> input = {1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0};
  int k = 2;

  // Create an instance of Solution class
  Solution sol;

  int length = sol.longestOnes(input, k);

  // Print the result
  cout << "Length of longest substring with at most " << k << " zeros: " << length << endl;

  return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
  /* Function to find the length of the
```

```
longest substring with at most k zeros */
int longestOnes(vector<int>& nums, int k) {

    // Length of the input array
    int n = nums.size();

    // Pointers for sliding window approach
    int l = 0, r = 0;

    /* Variables to count zeros
    and store maximum length*/
    int zeros = 0, maxLen = 0;

    /* Iterate through the array
    using sliding window approach*/
    while(r < n){
        if(nums[r] == 0){

            /* Increment zeros count
            when encountering a zero*/
            zeros++;
        }
        while(zeros > k){
            if(nums[l] == 0){

                /* Decrement zeros count
                when moving left pointer*/
                zeros--;
            }

            /* Move left pointer to the
```

```
            right to shrink the window*/
            l++;
        }

        /* Calculate the length
        of current substring*/
        int len = r - l + 1;

        /* Update maxLen if the current
        substring length is greater*/
        maxLen = max(maxLen, len);

        /* Move right pointer
        to expand the window*/
        r++;
      }

      // Return the maximum length
      return maxLen;
   }
};

int main() {
   vector<int> input = {1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0};
   int k = 2;

   // Create an instance of Solution class
   Solution sol;

   int length = sol.longestOnes(input, k);
```

```
    // Print the result
    cout << "Length of longest substring with at most " << k << " zeros: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the length of the
    longest substring with at most k zeros */
    int longestOnes(vector<int>& nums, int k) {

        // Length of the input array
        int n = nums.size();

        // Pointers for sliding window approach
        int l = 0, r = 0;

        /* Variables to count zeros
        and store maximum length */
        int zeros = 0, maxLen = 0;

        /* Iterate through the array
        using sliding window approach */
        while (r < n) {

            if(nums[r] == 0) zeros++;

            if (zeros > k) {
```

```
            if (nums[l] == 0) {

                /* Decrement zeros count
                when moving left pointer */
                zeros--;
            }
            l++;
        }
        if(zeros <= k){
            /* Calculate the length
            of current substring */
            int len = r - l + 1;

            /* Update maxLen if the current
            substring length is greater */
            maxLen = max(maxLen, len);
        }
        r++;
    }

    // Return the maximum length
    return maxLen;
  }
};

int main() {
  vector<int> input = {1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0};
  int k = 2;

  // Create an instance of Solution class
  Solution sol;
```

```
    int length = sol.longestOnes(input, k);

    // Print the result
    cout << "Length of longest substring with at most " << k << " zeros: " << length << endl;

    return 0;
}
```

Fruit Into Baskets

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the maximum
    fruits the basket can have */
    int totalFruits(vector<int>& fruits) {

        // Length of the input array
        int n = fruits.size();

        /* Variable to store the
        maximum length of substring*/
        int maxLen = 0;

        /* Iterate through all possible
        starting points of the substring*/
        for (int i = 0; i < n; i++) {

            /* Use unordered_set to track
            different types of fruits*/
```

```
        unordered_set<int> set;

        for (int j = i; j < n; j++) {

            // Add fruit type to the set
            set.insert(fruits[j]);

            /* Check if the number of different
            fruits is within the allowed limit*/
            if (set.size() <= 2) {

                /* Calculate the length
                of current substring*/
                int len = j - i + 1;

                maxLen = max(maxLen, len);
            } else break;
        }
    }

    // Return the maximum length;
    return maxLen;
  }
};

int main() {
  vector<int> input = {3, 3, 3, 1, 2, 1, 1, 2, 3, 3, 4};

  // Create an instance of Solution class
  Solution sol;
```

```
    int length = sol.totalFruits(input);

    // Print the result
    cout << "Maximum fruits in the basket is: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the maximum
    fruits the basket can have */
    int totalFruits(vector<int>& fruits) {

        // Length of the input array
        int n = fruits.size();

        /* Variable to store the
        maximum length of substring*/
        int maxLen = 0;

        /* Map to track the count of each
        fruit type in the current window*/
        unordered_map<int, int> mpp;

        // Pointers for the sliding window approach
        int l = 0, r = 0;

        while(r < n){
```

```
            mpp[fruits[r]]++;

            /* If number of different fruits exceeds
             2 shrink the window from the left*/
            if(mpp.size() > 2){
               while(mpp.size() > 2){
                  mpp[fruits[l]]--;
                  if(mpp[fruits[l]] == 0){
                     mpp.erase(fruits[l]);
                  }
                  l++;
               }
            }

            /* If number of different fruits
            is at most 2, update maxLen*/
            if(mpp.size() <= 2){
               maxLen = max(maxLen, r - l + 1);
            }

            r++;
         }

         // Return the maximum fruit
         return maxLen;
      }
};

int main() {
   vector<int> input = {3, 3, 3, 1, 2, 1, 1, 2, 3, 3, 4};
```

```
    // Create an instance of Solution class
    Solution sol;

    int length = sol.totalFruits(input);

    // Print the result
    cout << "Maximum fruits the can have is: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the maximum
    fruits the basket can have */
    int totalFruits(vector<int>& fruits) {

        // Length of the input array
        int n = fruits.size();

        /* Variable to store the
        maximum length of substring*/
        int maxLen = 0;

        /* Map to track the count of each
        fruit type in the current window*/
        unordered_map<int, int> mpp;

        // Pointers for the sliding window approach
```

```
        int l = 0, r = 0;

        while(r < n){
            mpp[fruits[r]]++;

            /* If number of different fruits exceeds
             2 shrink the window from the left*/
            if(mpp.size() > 2){
                mpp[fruits[l]]--;
                if(mpp[fruits[l]] == 0){
                    mpp.erase(fruits[l]);
                }
                l++;
            }

            /* If number of different fruits
            is at most 2, update maxLen*/
            if(mpp.size() <= 2){
                maxLen = max(maxLen, r - l + 1);
            }

            r++;
        }

        // Return the maximum fruit
        return maxLen;
    }
};

int main() {
    vector<int> input = {3, 3, 3, 1, 2, 1, 1, 2, 3, 3, 4};
```

```
    // Create an instance of Solution class
    Solution sol;

    int length = sol.totalFruits(input);

    // Print the result
    cout << "Maximum fruits the can have is: " << length << endl;

    return 0;
}
```

Longest Substring With At Most K Distinct Characters

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the maximum length of
    substring with at most k distinct characters */
    int kDistinctChar(string& s, int k) {

        /* Variable to store the
        maximum length of substring*/
        int maxLen = 0;

        /* Map to track the count of each
        character in the current window*/
        unordered_map<char, int> mpp;

        /* Iterate through each starting
        point of the substring*/
```

```
        for(int i = 0; i < s.size(); i++){
            // Clear map for a new starting point
            mpp.clear();

            for(int j = i; j < s.size(); j++){

                // Add the current character to the map
                mpp[s[j]]++;

                /* Check if the number of distinct
                characters is within the limit*/
                if(mpp.size() <= k){

                    /* Calculate the length of
                    the current valid substring*/
                    maxLen = max(maxLen, j - i + 1);
                }
                else break;
            }
        }

        // Return the maximum length found
        return maxLen;
    }
};

int main() {
    string s = "aaabbccd";
    int k = 2;

    // Create an instance of Solution class
```

```
    Solution sol;

    int length = sol.kDistinctChar(s, k);

    // Print the result
    cout << "Maximum length of substring with at most " << k << " distinct characters: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the length of the longest
    substring with at most k distinct characters*/
    int kDistinctChar(string& s, int k) {

        /* Initialize left pointer, right pointer,
        and maximum length of substring*/
        int l = 0, r = 0, maxLen = 0;

        // Hash map to store character frequencies
        unordered_map<char, int> mpp;

        while (r < s.size()) {

            // Increment frequency of current character
            mpp[s[r]]++;
```

```
        /* If the number of distinct characters
        exceeds k, shrink the window from the left*/
        while (mpp.size() > k) {

            // Decrement frequency of character at left pointer
            mpp[s[l]]--;
            if (mpp[s[l]] == 0) {

                /* Remove character from map
                if its frequency becomes zero*/
                mpp.erase(s[l]);
            }

            // Move left pointer to the right
            l++;
        }

        /* Update maximum length of substring with
        at most k distinct characters found so far*/
        if (mpp.size() <= k) {
            maxLen = max(maxLen, r - l + 1);
        }
        // Move right pointer
        r++;
    }
    // Return the maximum length found
    return maxLen;
  }
};

int main() {
```

```
    string s = "aaabbccd";

    // Create an instance of the Solution class
    Solution sol;

    int res = sol.kDistinctChar(s, 2);

    // Output the result
    cout << "The maximum length of substring with at most " << 2 << " distinct characters is: " << res
<< endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the maximum length of
    substring with at most k distinct characters */
    int kDistinctChar(string& s, int k) {

        // Length of the input string
        int n = s.size();

        /* Variable to store the
        maximum length of substring*/
        int maxLen = 0;

        /* Map to track the count of each
        character in the current window*/
```

```
        unordered_map<char, int> mpp;

        // Pointers for the sliding window approach
        int l = 0, r = 0;

        while(r < n){
            mpp[s[r]]++;

            /* If number of different characters exceeds
             k, shrink the window from the left*/
            if(mpp.size() > k){
                mpp[s[l]]--;
                if(mpp[s[l]] == 0){
                    mpp.erase(s[l]);
                }
                l++;
            }

            /* If number of different characters
            is at most k, update maxLen*/
            if(mpp.size() <= k){
                maxLen = max(maxLen, r - l + 1);
            }

            r++;
        }

        // Return the maximum length
        return maxLen;
    }
};
```

```
int main() {
    string s = "aaabbccd";
    int k = 2;

    // Create an instance of Solution class
    Solution sol;

    int length = sol.kDistinctChar(s, k);

    // Print the result
    cout << "Maximum length of substring with at most " << k << " distinct characters: " << length << endl;

    return 0;
}
```

Longest Repeating Character Replacement

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /*Function to find the longest substring
    with at most k characters replaced.*/
    int characterReplacement(string s, int k) {

        /* Variable to store the maximum
        length of substring found*/
        int maxLen = 0;

        // Iterate through each starting point of the substring
```

```
for (int i = 0; i < s.size(); ++i) {
    /* Variable to track the maximum frequency
    of any single character in the current window*/
    int maxFreq = 0;

    // Initialize hash array for character frequencies
    int hash[26] = {0};

    for (int j = i; j < s.size(); ++j) {

        /* Update frequency of current
        character in the hash array*/
        hash[s[j] - 'A']++;

        // Update max frequency encountered
        maxFreq = max(maxFreq, hash[s[j] - 'A']);

        // Calculate the number of changes needed to make
        int changes = (j - i + 1) - maxFreq;

        /* If the number of changes is less than or
        equal to k, the current window is valid*/
        if (changes <= k) {
            maxLen = max(maxLen, j - i + 1);
        }
        else break;

    }
}

/* Return the maximum length of substring
```

```
        with at most k characters replaced*/
        return maxLen;
    }
};

int main() {
    string s = "AABABBA";
    int k = 2;

    // Create an instance of Solution class
    Solution sol;

    int length = sol.characterReplacement(s, k);

    // Print the result
    cout << "Maximum length of substring with at most " << k << " characters replaced: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /*Function to find the longest substring
     with at most k characters replaced*/
    int characterReplacement(string s, int k) {

        /* Variable to store the maximum
        length of substring found*/
```

```
int maxLen = 0;

/* Variable to track the maximum frequency
of any single character in the current window*/
int maxFreq = 0;

//Pointers to maintain the current window [l, r]
int l = 0, r = 0;

// Hash array to count frequencies of characters
int hash[26] = {0};

// Iterate through each starting point of substring
while (r < s.size()) {

   /* Update frequency of current
   character in the hash array*/
   hash[s[r] - 'A']++;

   // Update max frequency encountered
   maxFreq = max(maxFreq, hash[s[r] - 'A']);

   // Check if current window is invalid
   while ((r - l + 1) - maxFreq > k) {

      /* Slide the left pointer to
      make the window valid again*/
      hash[s[l] - 'A']--;

      // Recalculate maxFreq for current window
      maxFreq = 0;
```

```
            for (int i = 0; i < 26; ++i) {
                maxFreq = max(maxFreq, hash[i]);
            }

            // Move left pointer forward
            l++;


        }

        /* Update maxLen with the length
        of the current valid substring*/
        maxLen = max(maxLen, r - l + 1);

        // Move right pointer forward to expand the window
        r++;
      }

      /* Return the maximum length of substring
      with at most k characters replaced*/
      return maxLen;
   }
};

int main() {
   string s = "AABABBA";
   int k = 2;

   // Create an instance of Solution class
   Solution sol;
```

```
    int length = sol.characterReplacement(s, k);

    // Print the result
    cout << "Maximum length of substring with at most " << k << " characters replaced: " << length << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /*Function to find the longest substring
     with at most k characters replaced*/
    int characterReplacement(string s, int k) {

        /* Variable to store the maximum
        length of substring found*/
        int maxLen = 0;

        /* Variable to track the maximum frequency
        of any single character in the current window*/
        int maxFreq = 0;

        //Pointers to maintain the current window [l, r]
        int l = 0, r = 0;

        // Hash array to count frequencies of characters
        int hash[26] = {0};
```

```
// Iterate through each starting point of substring
while (r < s.size()) {

    /* Update frequency of current
    character in the hash array*/
    hash[s[r] - 'A']++;

    // Update max frequency encountered
    maxFreq = max(maxFreq, hash[s[r] - 'A']);

    // Check if current window is invalid
    if ((r - l + 1) - maxFreq > k) {

        /* Slide the left pointer to
        make the window valid again*/
        hash[s[l] - 'A']--;

        // Move left pointer forward
        l++;



    }

    /* Update maxLen with the length
    of the current valid substring*/
    maxLen = max(maxLen, r - l + 1);

    // Move right pointer forward to expand the window
    r++;
}
```

```
        /* Return the maximum length of substring
        with at most k characters replaced*/
        return maxLen;
    }
};

int main() {
    string s = "AABABBA";
    int k = 2;

    // Create an instance of Solution class
    Solution sol;

    int length = sol.characterReplacement(s, k);

    // Print the result
    cout << "Maximum length of substring with at most " << k << " characters replaced: " << length << endl;

    return 0;
}
```

Minimum Window Substring

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /*Function to find the minimum length substring
    in string s that contains all characters from string t.*/
    string minWindow(string s, string t) {
```

```
/* Variable to store the minimum
length of substring found*/
int minLen = INT_MAX;

/* Variable to store the starting index
of the minimum length substring*/
int sIndex = -1;

// Iterate through string s
for (int i = 0; i < s.size(); ++i) {

   int hash[256] = {0};

   //Count the frquencies of characters in t.
   for (char c : t) {
      hash[c]++;
   }

   int count = 0;

   // Iterate through current window
   for (int j = i; j < s.size(); ++j) {
      if (hash[s[j]] > 0) {
         count++;
      }
      hash[s[j]]--;

      /* If all characters from t
      are found in current window*/
      if (count == t.size()) {
```

```
                /* Update minLen and sIndex
                if current window is smaller*/
                if (j - i + 1 < minLen) {
                    minLen = j - i + 1;
                    sIndex = i;
                }
                break;
            }
        }
    }

    // Return minimum length substring from s
    return (sIndex == -1) ? "" : s.substr(sIndex, minLen);
  }
};

int main() {
  string s = "ddaaabbca";
  string t = "abc";

  // Create an instance of Solution class
  Solution sol;

  string ans = sol.minWindow(s, t);

  // Print the result
  cout << "Minimum length substring containing all characters from \"" << t << "\" is: " << ans << endl;

  return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the minimum length
    substring in string s that contains
    all characters from string t. */
    string minWindow(string s, string t) {

        /* Variable to store the minimum
        length of substring found */
        int minLen = INT_MAX;

        /* Variable to store the starting index
        of the minimum length substring */
        int sIndex = -1;

        /* Array to count frequencies
        of characters in string t*/
        int hash[256] = {0};

        // Count the frequencies of characters in t
        for (char c : t) {
            hash[c]++;
        }

        int count = 0;
        int l = 0, r = 0;

        // Iterate through current window
```

```
    while (r < s.size()) {
        // Include the current character in the window
        if (hash[s[r]] > 0) {
            count++;
        }
        hash[s[r]]--;

        /* If all characters from t
        are found in current window */
        while (count == t.size()) {

            /* Update minLen and sIndex
            if current window is smaller */
            if (r - l + 1 < minLen) {
                minLen = r - l + 1;
                sIndex = l;
            }

            // Remove leftmost character from window
            hash[s[l]]++;
            if (hash[s[l]] > 0) {
                count--;
            }
            l++;
        }
        r++;
    }

    // Return minimum length substring from s
    return (sIndex == -1) ? "" : s.substr(sIndex, minLen);
}
```

```
};

int main() {
    string s = "ddaaabbca";
    string t = "abc";

    // Create an instance of Solution class
    Solution sol;

    string ans = sol.minWindow(s, t);

    // Print the result
    cout << "Minimum length substring containing all characters from \"" << t << "\" is: " << ans << endl;

    return 0;
}
```

Number of Substrings Containing All Three Characters

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the number of substrings
    containing all characters 'a', 'b', 'c' in string s. */
    int numberOfSubstrings(string s) {
        int count = 0;

        // Iterate through each starting point of substring
        for (int i = 0; i < s.size(); ++i) {
```

```
            // Array to track presence of 'a', 'b', 'c'
            int hash[3] = {0};

            // Iterate through each ending point of substring
            for (int j = i; j < s.size(); ++j) {

                // Mark current character in hash
                hash[s[j] - 'a'] = 1;

                /* Check if all characters
                'a', 'b', 'c' are present*/
                if (hash[0] + hash[1] + hash[2] == 3) {

                    // Increment count for valid substring
                    count++;
                }
            }
        }
        // Return the total count of substrings
        return count;
    }
};

int main() {
    string s = "bbacba";

    // Create an instance of Solution class
    Solution sol;

    int ans = sol.numberOfSubstrings(s);
```

```
    // Print the result
    cout << "Number of substrings containing 'a', 'b', 'c' in \"" << s << "\" is: " << ans << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the number of substrings
    containing all characters 'a', 'b', 'c' in string s. */
    int numberOfSubstrings(string s) {

        /* Array to store the last seen
        index of characters 'a', 'b', 'c'*/
        int lastSeen[3] = {-1, -1, -1};

        int count = 0;

        // Iterate through each character in string s
        for (int i = 0; i < s.size(); ++i) {

            // Update lastSeen index for
            lastSeen[s[i] - 'a'] = i;

            /* Check if all characters 'a',
            'b', 'c' have been seen*/
            if (lastSeen[0] != -1 && lastSeen[1] != -1 && lastSeen[2] != -1) {

                /* Count valid substrings
```

```
            ending at current index*/
            count += 1 + min({lastSeen[0], lastSeen[1], lastSeen[2]});
        }
    }

    // Return total count of substrings
    return count;
  }
};

int main() {
  string s = "bbacba";

  // Create an instance of Solution class
  Solution sol;

  int ans = sol.numberOfSubstrings(s);

  // Print the result
  cout << "Number of substrings containing 'a', 'b', 'c' in \"" << s << "\" is: " << ans << endl;

  return 0;
}
```

Binary Subarrays With Sum

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
  /* Function to find the number of
  subarrays with sum equal to `goal`*/
```

```
    int numSubarraysWithSum(vector<int>& nums, int goal) {

        /*Calculate the number of subarrays with
        sum exactly equal to `goal` by using the
        difference between subarrays with sum less
        than or equal to `goal` and those with sum
        less than or equal to `goal-1`*/
        return numSubarraysWithSumLessEqualToGoal(nums, goal) -
numSubarraysWithSumLessEqualToGoal(nums, goal - 1);
    }

private:
    /* Helper function to find the number of
    subarrays with sum less than or equal to `goal`*/
    int numSubarraysWithSumLessEqualToGoal(vector<int>& nums, int goal) {

        /* If goal is negative, there
        can't be any valid subarray sum*/
        if (goal < 0) return 0;

        //Pointers to maintain the current window
        int l = 0, r = 0;

        /* Variable to track the current
        sum of elements in the window*/
        int sum = 0;

        // Variable to count the number of subarrays
        int count = 0;

        /* Iterate through the array
```

```
        using the right pointer `r`*/
        while (r < nums.size()) {
            sum += nums[r];

            /* Shrink the window from the left
            side if the sum exceeds `goal`*/
            while (sum > goal) {
                sum -= nums[l];

                // Move the left pointer `l` forward
                l++;
            }

            /* Count all subarrays ending at
            `r` that satisfy the sum condition*/
            count += (r - l + 1);

            // Move the right pointer `r` forward
            r++;
        }

        // Return the total count of subarrays
        return count;
    }
};

int main() {
    vector<int> nums = {1, 0, 0, 1, 1, 0};
    int goal = 2;

    // Create an instance of Solution class
```

```
    Solution sol;

    int ans = sol.numSubarraysWithSum(nums, goal);

    // Print the result
    cout << "Number of substrings with sum \"" << goal << "\" is: " << ans << endl;

    return 0;
}
```

Count number of Nice subarrays

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    /* Function to find the number of subarrays
    nice subarrays with k odd numbers*/
    int numberOfOddSubarrays(vector<int>& nums, int k) {

        /*Calculate the number of subarrays with
        sum exactly equal to k by using the
        difference between subarrays with sum less
        than or equal to k and those with sum
        less than or equal to k-1*/
        return helper(nums, k) - helper(nums, k - 1);
    }

private:
    /* Helper function to find the number of
    subarrays with sum less than or equal to k*/
    int helper(vector<int>& nums, int k) {
```

```
/* If goal is negative, there
can't be any valid subarray sum*/
if (k < 0) return 0;

//Pointers to maintain the current window
int l = 0, r = 0;

/* Variable to track the current
sum of elements in the window*/
int sum = 0;

// Variable to count the number of subarrays
int count = 0;

/* Iterate through the array
using the right pointer `r`*/
while (r < nums.size()) {
   sum = sum + nums[r] % 2;

   /* Shrink the window from the left
   side if the sum exceeds k*/
   while (sum > k) {
      sum = sum -  nums[l] % 2;

      // Move the left pointer `l` forward
      l++;
   }

   /* Count all subarrays ending at
   `r` that satisfy the sum condition*/
```

```
            count += (r - l + 1);

            // Move the right pointer `r` forward
            r++;
        }

        // Return the total count of subarrays
        return count;
    }
};

int main() {
    vector<int> nums = {1, 1, 2, 1, 1};
    int k = 1;

    // Create an instance of Solution class
    Solution sol;

    int ans = sol.numberOfOddSubarrays(nums, k);

    // Print the result
    cout << "Number of nice substrings with \"" << k << "\" odd numbers is: " << ans << endl;

    return 0;
}
```

Solution

Introduction to Stack and Queue Data Structures

Stack:

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle. This means that the last element added to the stack will be the first one to be removed. Stacks are analogous to a stack of plates where you can only add or remove plates from the top. Common operations in a stack include:

- **Push:** Adding an element to the top of the stack.
- **Pop:** Removing the top element from the stack.
- **Peek/Top:** Retrieving the top element without removing it.

Stacks are used in various applications such as reversing a word, backtracking algorithms (like finding a path in a maze), and in the implementation of function calls in recursion.

Queue:

A queue is a linear data structure that follows the First In, First Out (FIFO) principle. This means that the first element added to the queue will be the first one to be removed. Queues are similar to a line of people waiting for a service where the first person in line is the first to be served. Common operations in a queue include:

- **Enqueue:** Adding an element to the end of the queue.
- **Dequeue:** Removing the front element from the queue.
- **Front/Peek:** Retrieving the front element without removing it.

Queues are widely used in scenarios like scheduling processes in operating systems, handling requests in web servers, and in breadth-first search algorithms.

Understanding stacks and queues is fundamental in computer science as they provide the basis for more complex data structures and algorithms.

Watch the video for explanation, and to practice and read editorials around these problems, use the given below links.

1. Implement Stack using Arrays
2. Implement Queue using Arrays
3. Implement Stack using Queue
4. Implement Queue using Stack
5. Implement stack using Linkedlist
6. Implement queue using Linkedlist

21

Count number of Nice subarrays - TUF+

Implement Stack using Arrays

```
#include <bits/stdc++.h>

using namespace std;


class ArrayStack {

private:
```

```
    // Array to hold elements
    int* stackArray;
    // Maximum capacity
    int capacity;
     // Index of top element
    int topIndex;

public:
    // Constructor
    ArrayStack(int size = 1000) {
        capacity = size;
        stackArray = new int[capacity];
        // Initialize stack as empty
        topIndex = -1;
    }

    // Destructor
    ~ArrayStack() {
        delete[] stackArray;
    }

    // Pushes element x
    void push(int x) {
        if (topIndex >= capacity - 1) {
            cout << "Stack overflow" << endl;
            return;
        }
        stackArray[++topIndex] = x;
    }

    // Removes and returns top element
```

```
    int pop() {
        if (isEmpty()) {
            cout << "Stack is empty" << endl;
            // Return invalid value
            return -1;
        }
        return stackArray[topIndex--];
    }

    // Returns top element
    int top() {
        if (isEmpty()) {
            cout << "Stack is empty" << endl;
            return -1;
        }
        return stackArray[topIndex];
    }

   /* Returns true if the
   stack is empty, false otherwise*/
    bool isEmpty() {
        return topIndex == -1;
    }
};

// Main Function
int main() {
    ArrayStack stack;
    vector<string> commands = {"ArrayStack", "push", "push", "top", "pop", "isEmpty"};
    vector<vector<int>> inputs = {{}, {5}, {10}, {}, {}, {}};
```

```
    for (size_t i = 0; i < commands.size(); ++i) {
        if (commands[i] == "push") {
            stack.push(inputs[i][0]);
            cout << "null ";
        } else if (commands[i] == "pop") {
            cout << stack.pop() << " ";
        } else if (commands[i] == "top") {
            cout << stack.top() << " ";
        } else if (commands[i] == "isEmpty") {
            cout << (stack.isEmpty() ? "true" : "false") << " ";
        } else if (commands[i] == "ArrayStack") {
            cout << "null ";
        }
    }

    return 0;
}
```

Implement Queue using Arrays

```
#include <bits/stdc++.h>
using namespace std;

// Class implementing Queue using Arrays
class ArrayQueue {
    // Array to store queue elements
    int* arr;
    // Indices for start and end of the queue
    int start, end;
    // Current size and maximum size of the queue
    int currSize, maxSize;

public:
```

```
// Constructor
ArrayQueue() {
    arr = new int[10];
    start = -1;
    end = -1;
    currSize = 0;
    maxSize = 10;
}

// Method to push an element into the queue
void push(int x) {
    // Check if the queue is full
    if (currSize == maxSize) {
        cout << "Queue is full\nExiting..." << endl;
        exit(1);
    }

    // If the queue is empty, initialize start and end
    if (end == -1) {
        start = 0;
        end = 0;
    }
    else {
        // Circular increment of end
        end = (end + 1) % maxSize;
    }

    arr[end] = x;
    currSize++;
}
```

```
// Method to pop an element from the queue
int pop() {
    // Check if the queue is empty
    if (start == -1) {
        cout << "Queue Empty\nExiting..." << endl;
        exit(1);
    }
    int popped = arr[start];

    // If the queue has only one element, reset start and end
    if (currSize == 1) {
        start = -1;
        end = -1;
    }
    else {
        // Circular increment of start
        start = (start + 1) % maxSize;
    }

    currSize--;
    return popped;
}

// Method to get the front element of the queue
int peek() {
    // Check if the queue is empty
    if (start == -1) {
        cout << "Queue is Empty" << endl;
        exit(1);
    }
    return arr[start];
```

```
    }

    // Method to determine whether the queue is empty
    bool isEmpty() {
        return (currSize == 0);
    }
};

int main() {
    ArrayQueue queue;
    vector<string> commands = {"ArrayQueue", "push", "push",
                               "peek", "pop", "isEmpty"};
    vector<vector<int>> inputs = {{}, {5}, {10}, {}, {}, {}};

    for (int i = 0; i < commands.size(); ++i) {
        if (commands[i] == "push") {
            queue.push(inputs[i][0]);
            cout << "null ";
        } else if (commands[i] == "pop") {
            cout << queue.pop() << " ";
        } else if (commands[i] == "peek") {
            cout << queue.peek() << " ";
        } else if (commands[i] == "isEmpty") {
            cout << (queue.isEmpty() ? "true" : "false") << " ";
        } else if (commands[i] == "ArrayQueue") {
            cout << "null ";
        }
    }

    return 0;
}
```

Implement Stack using Queue

```
#include <bits/stdc++.h>
using namespace std;

// Stack implementation using Queue
class QueueStack {
    // Queue
    queue<int> q;

public:
    // Method to push element in the stack
    void push(int x) {
        // Get size
        int s = q.size();
        // Add element
        q.push(x);

        // Move elements before new element to back
        for (int i = 0; i < s; i++) {
            q.push(q.front());
            q.pop();
        }
    }

    // Method to pop element from stack
    int pop() {
        // Get front element
        int n = q.front();
        // Remove front element
        q.pop();
        // Return removed element
```

```
        return n;
    }

    // Method to return the top of stack
    int top() {
        // Return front element
        return q.front();
    }

    // Method to check if the stack is empty
    bool isEmpty() {
        return q.empty();
    }
};

int main() {
    QueueStack st;

    // List of commands
    vector<string> commands = {"QueueStack", "push", "push",
                               "pop", "top", "isEmpty"};
    // List of inputs
    vector<vector<int>> inputs = {{}, {4}, {8}, {}, {}, {}};

    for (int i = 0; i < commands.size(); ++i) {
        if (commands[i] == "push") {
            st.push(inputs[i][0]);
            cout << "null ";
        } else if (commands[i] == "pop") {
            cout << st.pop() << " ";
        } else if (commands[i] == "top") {
```

```
            cout << st.top() << " ";
        } else if (commands[i] == "isEmpty") {
            cout << (st.isEmpty() ? "true" : "false") << " ";
        } else if (commands[i] == "QueueStack") {
            cout << "null ";
        }
    }

    return 0;
}
```

mplement Queue using Stack

```
#include <bits/stdc++.h>
using namespace std;

// Queue implementation using stack
class StackQueue {
private:
    stack <int> st1, st2;

public:
    // Empty Constructor
    StackQueue () {

    }

    // Method to push elements in the queue
    void push(int x) {
        /* Pop out elements from the first stack
        and push on top of the second stack */
        while (!st1.empty()) {
            st2.push(st1.top());
```

```
        st1.pop();
    }

    // Insert the desired element
    st1.push(x);

    /* Pop out elements from the second stack
    and push back on top of the first stack */
    while (!st2.empty()) {
        st1.push(st2.top());
        st2.pop();
    }
}

// Method to pop element from the queue
int pop() {
    // Edge case
    if (st1.empty()) {
        cout << "Stack is empty";
        return -1; // Representing empty stack
    }

    // Get the top element
    int topElement = st1.top();
    st1.pop(); // Perform the pop operation

    return topElement; // Return the popped value
}

// Method to get the front element from the queue
int peek() {
```

```
        // Edge case
        if (st1.empty()) {
            cout << "Stack is empty";
            return -1; // Representing empty stack
        }

        // Return the top element
        return st1.top();
    }

    // Method to find whether the queue is empty
    bool isEmpty() {
        return st1.empty();
    }
};

int main() {
    StackQueue q;

    // List of commands
    vector<string> commands = {"StackQueue", "push", "push",
                               "pop", "peek", "isEmpty"};
    // List of inputs
    vector<vector<int>> inputs = {{}, {4}, {8}, {}, {}, {}};

    for (int i = 0; i < commands.size(); ++i) {
        if (commands[i] == "push") {
            q.push(inputs[i][0]);
            cout << "null ";
        } else if (commands[i] == "pop") {
            cout << q.pop() << " ";
```

```
        } else if (commands[i] == "peek") {
            cout << q.peek() << " ";
        } else if (commands[i] == "isEmpty") {
            cout << (q.isEmpty() ? "true" : "false") << " ";
        } else if (commands[i] == "StackQueue") {
            cout << "null ";
        }
    }

    return 0;
}
```

Implement stack using Linkedlist

```
#include <bits/stdc++.h>
using namespace std;

// Node structure
struct Node {
    int val;
    Node *next;
    Node(int d) {
        val = d;
        next = NULL;
    }
};

// Structure to represent stack
class LinkedListStack {
private:
    Node *head; // Top of Stack
    int size; // Size
```

```
public:
    // Constructor
    LinkedListStack() {
        head = NULL;
        size = 0;
    }

    // Method to push an element onto the stack
    void push(int x) {
        // Creating a node
        Node *element = new Node(x);

        element->next = head; // Updating the pointers
        head = element; // Updating the top

        // Increment size by 1
        size++;
    }

    // Method to pop an element from the stack
    int pop() {
        // If the stack is empty
        if (head == NULL) {
            return -1; // Pop operation cannot be performed
        }

        int value = head->val; // Get the top value
        Node *temp = head; // Store the top temporarily
        head = head->next; // Update top to next node
        delete temp; // Delete old top node
        size--; // Decrement size
```

```
        return value; // Return data
    }

    // Method to get the top element of the stack
    int top() {
        // If the stack is empty
        if (head == NULL) {
            return -1; // Top element cannot be accessed
        }

        return head->val; // Return the top
    }

    // Method to check if the stack is empty
    bool isEmpty() {
        return (size == 0);
    }
};

int main() {
    // Creating a stack
    LinkedListStack st;

    // List of commands
    vector<string> commands = {"LinkedListStack", "push", "push",
                               "pop", "top", "isEmpty"};
    // List of inputs
    vector<vector<int>> inputs = {{}, {3}, {7}, {}, {}, {}};

    for (int i = 0; i < commands.size(); ++i) {
```

```
        if (commands[i] == "push") {
            st.push(inputs[i][0]);
            cout << "null ";
        } else if (commands[i] == "pop") {
            cout << st.pop() << " ";
        } else if (commands[i] == "top") {
            cout << st.top() << " ";
        } else if (commands[i] == "isEmpty") {
            cout << (st.isEmpty() ? "true" : "false") << " ";
        } else if (commands[i] == "LinkedListStack") {
            cout << "null ";
        }
    }

    return 0;
}
```

Implement queue using Linkedlist

```
#include <bits/stdc++.h>
using namespace std;

// Node structure
struct Node {
    int val;
    Node *next;
    Node(int d) {
        val = d;
        next = NULL;
    }
};

// Structure to represent stack
```

```
class LinkedListQueue {
private:
   Node *start; // Start of the queue
   Node *end; // End of the queue
   int size; // Size of the queue

public:
   // Constructor
   LinkedListQueue() {
      start = end = NULL;
      size = 0;
   }

   // Method to push an element in the queue
   void push(int x) {
      // Creating a node
      Node *element = new Node(x);

      // If it is the first element being pushed
      if(start == NULL) {
         // Initialise the pointers
         start = end = element;
      }
      else {
         end->next = element; // Updating the pointers
         end = element; // Updating the end
      }

      // Increment size by 1
      size++;
   }
```

```
// Method to pop an element from the queue
int pop() {
   // If the queue is empty
   if (start == NULL) {
      return -1; // Pop operation cannot be performed
   }

   int value = start->val; // Get the front value
   Node *temp = start; // Store the front temporarily
   start = start->next; // Update front to next node
   delete temp; // Delete old front node
   size--; // Decrement size

   if(start == NULL){
      end = NULL;
   }

   return value; // Return data
}

// Method to get the front element in the queue
int peek() {
   // If the stack is empty
   if (start == NULL) {
      return -1; // Top element cannot be accessed
   }

   return start->val; // Return the top
}
```

```
    // Method to check if the queue is empty
    bool isEmpty() {
        return (size == 0);
    }
};

int main() {
    // Creating a queue
    LinkedListQueue q;

    // List of commands
    vector<string> commands = {"LinkedListQueue", "push", "push",
                               "peek", "pop", "isEmpty"};
    // List of inputs
    vector<vector<int>> inputs = {{}, {3}, {7}, {}, {}, {}};

    for (int i = 0; i < commands.size(); ++i) {
        if (commands[i] == "push") {
            q.push(inputs[i][0]);
            cout << "null ";
        } else if (commands[i] == "pop") {
            cout << q.pop() << " ";
        } else if (commands[i] == "peek") {
            cout << q.peek() << " ";
        } else if (commands[i] == "isEmpty") {
            cout << (q.isEmpty() ? "true" : "false") << " ";
        } else if (commands[i] == "LinkedListQueue") {
            cout << "null ";
        }
    }
```

```
    return 0;
}
```

Balanced Paranthesis

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    /* Function to match the opening
    and closing brackets */
    bool isMatched(char open, char close) {

        // Match
        if((open == '(' && close == ')') ||
          (open == '[' && close == ']') ||
          (open == '{' && close == '}')
        )
        return true;

        // Mismatch
        return false;
    }

public:
    /* Function to check if the
    string is valid */
    bool isValid(string str) {

        // Initialize a stack
        stack<char> st;
```

```
        // Start iterating on the string
        for(int i=0; i < str.length(); i++) {

            // If an opening bracket is found
            if(str[i] == '(' || str[i] == '[' ||
               str[i] == '{') {
                // Push on top of stack
                st.push(str[i]);
            }

            // Else if a closing bracket is found
            else {
                // Return false if stack is empty
                if(st.empty()) return false;

                // Get the top elemenfrom stack
                char ch = st.top();
                st.pop();

                /* If the opening and closing brackets
                are not matched, return false */
                if(!isMatched(ch, str[i]))
                    return false;
            }
        }

        /* If stack is empty, the
        string is valid, else invalid */
        return st.empty();
    }
};
```

```
int main() {
    string str = "()[{}()]";

    /* Creating an instance of
    Solution class */
    Solution sol;

    /* Function call to check if the
    string is valid */
    bool ans = sol.isValid(str);

    if(ans)
        cout << "The given string is valid.";
    else
        cout << "The given string is invalid.";

    return 0;
}
```